

Graphs

[03graphs.pdf](#)

这一章的核心在于建模思维，图其实是一种统一的建模语言，现实中的很多问题虽然表面上看着不尽相同，但其实都是把对象看成点、关系看成边，从而转化成了图问题。

最基本的无向图记作：

$$G = (V, E)$$

其中：

- V 是顶点集合，也就是点；
- E 是边集合，也就是点与点之间的连接关系。

PPT 还专门强调了两个规模参数：

$$n = |V|, \quad m = |E|$$

以后分析图算法复杂度时，几乎都会写成 $O(m + n)$ 、 $O(n^2)$ 这种形式，所以一定要记住： n 表示点数， m 表示边数。

图的两种存储方式：

1. 邻接矩阵

邻接矩阵是一个 $n \times n$ 的矩阵。若 (u, v) 是边，就令

$$A_{uv} = 1$$

否则为 0。PPT 给出的特点是：

- 空间复杂度是 $\Theta(n^2)$ ；
- 判断 (u, v) 是否为边，只需 $\Theta(1)$ ；
- 但如果想找出所有边，就需要 $\Theta(n^2)$ 时间。

这说明邻接矩阵的优点是“查某一条边很快”，缺点是“很占空间”。

2. 邻接表

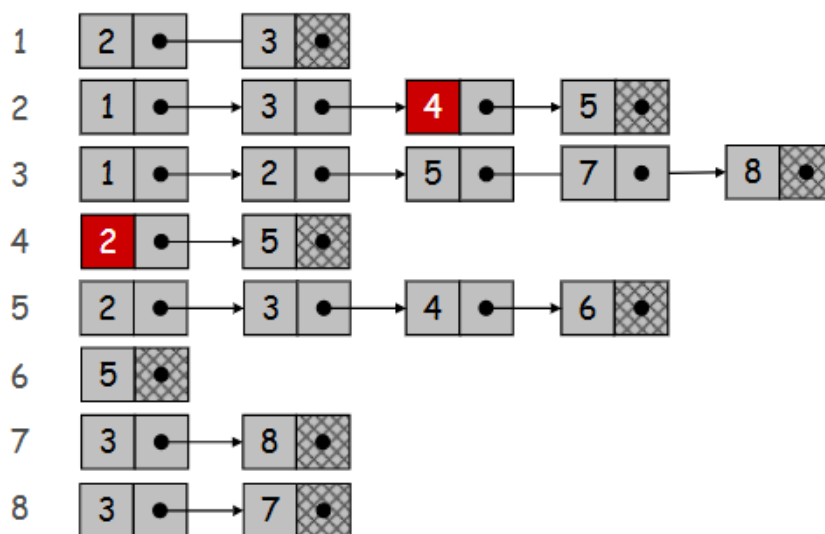
邻接表可以理解为：给每个点配一个链表或数组，里面存它所有的邻居。PPT 说它的特点是：

- 空间复杂度是 $\Theta(m + n)$;
- 判断 (u, v) 是否是边，需要在 u 的邻居表里找，所以一般是 $O(\deg(u))$;
- 但如果想遍历所有边，只需 $\Theta(m + n)$ 。

这里的 $\deg(u)$ 表示点 u 的度，也就是它的邻居个数。不占空间，但是查某一条边比较慢

(邻接矩阵)

	1	2	3	4	5	6	7	8
1	0	1	1	0	0	0	0	0
2	1	0	1	1	0	0	0	0
3	1	1	0	0	1	0	1	1
4	0	1	0	1	1	0	0	0
5	0	1	1	1	0	1	0	0
6	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	1
8	0	0	1	0	0	0	1	0



(邻接表：数组 + 链表)

路径、联通、环与树

什么是路径，什么又是简单路径？

路径是一个点序列

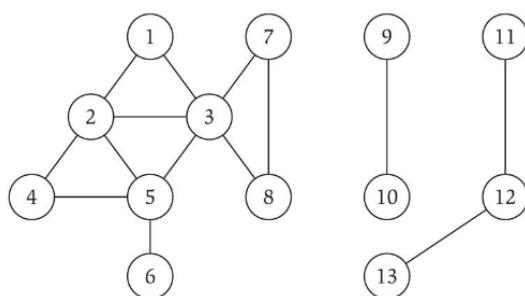
$$v_1, v_2, \dots, v_k$$

并且每一对相邻点 v_i, v_{i+1} 之间都存在边。直观地说，路径就是“沿边一步一步走过去的一条路线”。特别地，如果路径中的所有点都不互相重复，那么这就是一个**简单路径**。

连通是什么？

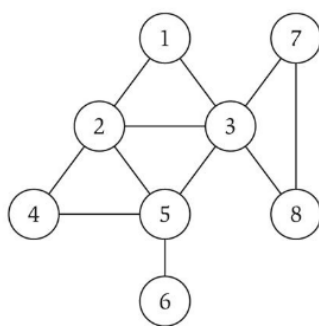
一个无向图如果对任意两个点 u, v ，都存在一条从 u 到 v 的路径，那么这个图就是连通的。可以把“连通”理解成：**整个图是一整块，没有被分裂成几个互不来往的小岛。**

我们以一个点作为起点，然后进行一次完整的DFS或者BFS，就可以找到这个起点所在的连通。



环是什么？

环是首尾相接的一条路径，要求起点和终点是同一个点，并且中间点都不重复。环就是“绕一圈又回到原点”。



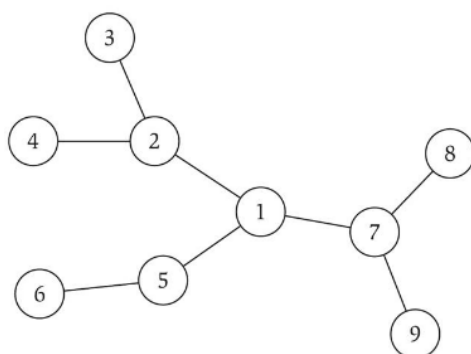
cycle $C = 1-2-4-5-3-1$

什么样的图可以称之为“树”？

如果一个**无向图**，**联通且无环**，那么就可以称之为是树，并且有这样的性质:对一个有 n 个点的无向图，下面三个条件中任意两个可以推出第三个：

1. 图连通;
2. 图无环;
3. 图有 $n - 1$ 条边。

一定要牢记这几条性质，我们能不能判断某个图是树就要看这些了！



图遍历：

从这里开始正式进入图算法

图的广度优先遍历→BFS：

从起点 s 出发，一层一层向外扩张。对每个 i ，第 i 层 L_i 恰好由所有与 s 的距离正好等于 i 的点组成。并且，当且仅当点 t 出现在某一层中时， s 到 t 存在路径。

这句话很重要。它说明：

- BFS 不只是能判断“能不能到”；
- BFS 还能直接给出**无权图中的最短距离**。

其实直观理解也很方便，无权图中每一个边都是平权的，所以边数就是路径长度，差几层就是差几个边嘛。并且BFS还有一个重要的逻辑：

若 T 是图 G 的一棵 BFS 树，而 (x, y) 是图中的任一条边，那么 x 和 y 的层数至多相差 1。**[意思就是相邻的两个点，在BFS中最多差一层]**

关于时间复杂度，容易看出的上界是 $O(n^2)$ ，但接着说明，若图用邻接表存储，BFS 实际上是

$$O(m + n)$$

原因是：

- 每个点只会进入某一层一次，所以点的处理总量是 $O(n)$ ；

- 处理某个点 u 时，只会检查它的 $\deg(u)$ 条 incident edges;
- 所有点合起来，总边检查次数是

$$\sum_{u \in V} \deg(u) = 2m$$

因为无向图中每条边恰好会在两个端点各被数一次。

图遍历里，若每个点处理一次、每条边处理常数次数，总复杂度通常就是 $O(m + n)$ 。

二分图与奇环：

什么是二分图呢？

无向图 $G = (V, E)$ 若能把所有点染成红、蓝两色，并使得每条边的两个端点颜色都不同，则称 G 是二分图。

很多图问题在二分图上会变得更容易，比如 matching；有些原本困难的问题，在二分图上会变得 tractable。也就是说，二分性是一种非常有用的结构信息。（比如稳定匹配 stable-matching，就可以借助二分图）

什么是奇环？

顾名思义长度为奇数的环。

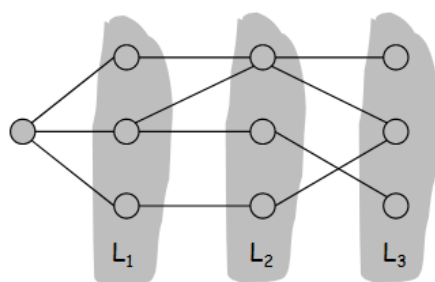
性质

如果图是二分图，那他不可能有奇长度的环。原因也很简单→沿着环走，颜色必须红蓝红蓝交替变化。若环长是奇数，走一圈回到起点时，颜色会冲突，不可能完成二染色。

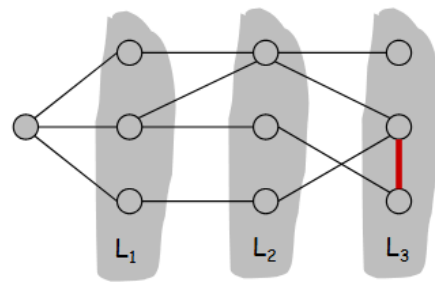
BFS分层与二分图的关系：

设 G 是连通图，从某点 s 做 BFS，得到层 $L_0, \dots, L_k$ 。则恰有两种情况之一发生：

1. 没有边连接同一层的两个点，此时图是二分图；
2. 存在一条边连接同一层的两个点，此时图中存在奇环，因此不是二分图。



Case (i)



Case (ii)

PPT很巧妙的证明了，在同层有连接的情况下，会出现奇环。

一个完整的链条就是：

二分图 $\iff$ 可二染色 $\iff$ 无奇环 $\iff$ BFS分层中无同层边

有向图与强连通：

在有向图中，“能从 u 到 v ”和“能从 v 到 u ”是两件不同的事。

- 若从 u 能到 v ，且从 v 也能到 u ，则 u, v **mutually reachable**；
- 若图中每一点都互相可达，则图是**strongly connected**。

“连通”是无向图概念，“强连通”是有向图里的对应概念，而且要求更强，因为方向不能反着随便走。

强连通判定算法：

我们给出一个非常经典的 $O(m + n)$ 算法：

1. 任取一个点 s ；
2. 在原图 G 中从 s 做一次 BFS；
3. 在反图 G^{rev} 中从 s 再做一次 BFS；
4. 当且仅当两次 BFS 都能到达所有点时，图强连通。

这里的反图 G^{rev} 指的是：**把原图每条边的方向全部反过来**。因为在反图里从 s 能走到某个点 u ，就等价于在原图里 u 能走到 s 。

💡 Tip

对于一个图，如果做了上面的操作，也就是正反都可以建立 s 与任何其他点的联系，我们就不要想着说： s 万一是个特殊的点呢？万一只有 s 能成立呢？

我们应当发现 s 可以当作一个中介点，也就是说如果对任意点 u, v ，都有 $u \rightarrow s, s \rightarrow v$ ，那么就能把这两条路径拼起来，得到：

$$u \rightarrow s \rightarrow v$$

也就是说，任意 u 都能到任意 v

(我原本想的是每个点全做一次BFS，这样的话如果每个点都能到其他所有点，那肯定是个强连通，但这个想法虽然思路正确，在算力和复杂度上实在过于浪费，上述方法只需两次BFS，直接省下 n 倍复杂度！)

DAG与拓扑排序

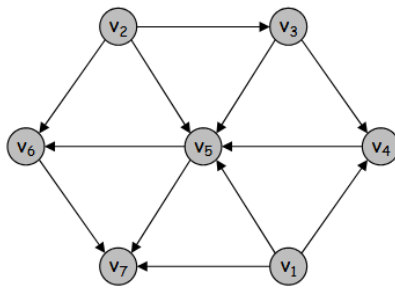
DAG 是 Directed Acyclic Graph 的缩写，意思是有向无环图。PPT 定义：有向图的一个拓扑序，是把所有点排成

$$v_1, v_2, \dots, v_n$$

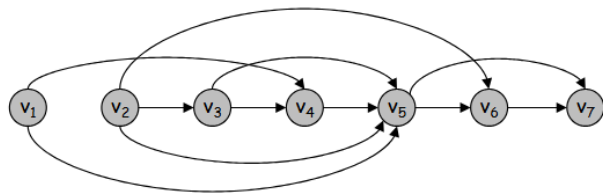
这样一个顺序，使得对每条边 (v_i, v_j) ，都有

$$i < j$$

也就是说，所有箭头都只能从前往后指。



a DAG



a topological ordering

为什么“有拓扑序”就一定“没有有向环”？

其实很好证嘛，有环不就是走了一圈又回到了原本出发的地方，也就是节点的下标又回到了出发的下标，但是拓扑排序的下标严格递增得，不可能回去。

为什么每个DAG都有入度为0的点？并且都有拓扑排序呢？

- 假设 DAG 中每个点的入度都至少为 1；
- 从任意点 v 出发，不断沿着“入边”往回走；（既然假设了入度至少唯一，那么说明每一个点

都一定有入边，那么会无限走下去)

- 由于点数有限，走着走着必然会重复访问某个点；
- 于是形成一个有向环；
- 这与 DAG“无环”矛盾。

都有拓扑排序证明：

这里采用数学归纳法证明：

- 若只有 1 个点，显然成立；
- 对 $n > 1$ 的 DAG，先找一个入度为 0 的点 v ；
- 删掉 v 后，剩余图仍然是 DAG；

Note

$G - v$ 仍然是 DAG，**因为删除 v 不可能创造出新的环**。拓扑排序中特别删入度为 0 的点，是因为这样的点可以合法放在序列最前面。

被删的点有两种可能，一种是它指向的点也只被它指向，那么删除改点后，他指向的点就变成了入度为 0，构成归纳性问题。第二种可能是他指向的点同时也被别的节点指向，那么顺着那条路也一定可以找到一个入度为 0 的节点，那么删除后，依然保持 DAG，故已构成归纳性问题。

- 由归纳假设，剩余图有拓扑排序；
- 把 v 放在最前面，再接上剩余图的拓扑序，就得到整个图的拓扑序。

因此：

DAG $\Rightarrow$ 有拓扑序

图可以拓扑排序 $\iff$ 它是 DAG

拓扑排序的时间复杂度是 $O(m + n)$ ，这和邻接表遍历复杂度一样，和 BFS 遍历的复杂度一样，因为它们本质上全都是：**每个节点访问一遍，每个边只做常数级别处理**。

此处要注意一下：

DFS 复杂度取决于图的存储方式：邻接表是 $O(m + n)$ ，邻接矩阵是 $O(n^2)$ 。

Note

小测试，学了这些内容后，如果有一道题，让你设计一个算法，来检测一个有向图是否有环，我们该怎么做？

拓扑排序法：

对该图进行一次拓扑排序，看能不能将所有节点输出，如果可以，则无环